

Milestone 2 Report

CMPT310 - D200

Group 20

Abraham Mormontoy	aem19@sfu.ca
-------------------	--------------

Aly Soliman	asa516@sfu.ca
-------------	---------------

Karan Passi	kpa77@sfu.ca
-------------	--------------

Viktor Zepeda	vza12@sfu.ca
---------------	--------------

Project Summary	1
Significant Accomplishments	2
Dataset Preparation	2
Model Design	2
Training and Results	3
Proof of accomplishment	3
Per-class accuracy comparison	3
Model Training Output	5
Benchmark Breeds test results	6
Challenges	6
Project Changes	7
Appendix A: Chosen Dog Breeds	7
Appendix B: Image Transform Pipelines	8

Project Summary

The goal of this project is to develop an AI model capable of identifying and categorizing 50 dog breeds to help users make more informed decisions for their pets' health. We use transfer learning on torch vision's ResNet-18 Convolutional Neural Network and replaced the final layer with our selected dog breeds. The model takes formatted JPG image inputs and outputs the 3 most likely breeds for the given dog.

Significant Accomplishments

Dataset Preparation

We narrowed down the list to 50 dog breeds, which are listed in appendix A. The Stanford Dogs dataset prefixes every breed folder with an ID in the format nXXXXXXXXX-, which we removed using a cleanup script. This was done because PyTorch's ImageFolder uses the folder name directly as the class label. We then used another script to cut down each breed's folder to contain exactly 150 images to avoid issues related to class imbalance.

The dataset is loaded using PyTorch's torchvision.datasets.ImageFolder. This scans the selected breeds' root directory, treats each immediate subfolder as one class, and then builds a list of (image path, class index) pairs automatically. Class indices are assigned alphabetically: index 0 = first breed alphabetically, index 49 = last. This mapping is stored in class_names and is used to translate predictions back to breed names.

The full dataset is split into 75% training, 25% test using PyTorch's random_split. We also used a fixed random seed (42) to ensure that the split is identical across every run. We chose to perform the split ourselves rather than use the Stanford-provided split files, because it gave us direct control over the ratio and avoided introducing the Stanford test set into our training data. We considered using k-fold cross validation as is described in the lecture material, however, due to the relatively small number of images available for each dog breed, we decided that the benefit would not be worthwhile.

Finally, before being fed into the model, the images all pass through a transform pipeline which was implemented using torchvision.transforms.Compose. The full pipeline is detailed in Appendix B.

Model Design

We use ResNet-18 as our base, which is a CNN from torch vision's model library that has 18 layers. This model was originally trained on the ImageNet dataset, which holds 1.2 million images spanning 1000 classes, some of which are dog related categories.

We adapted this model to fit our use case by freezing all the base layers and then replacing the final output layer with our own. This approach was suggested to us during earlier feedback, and we chose to implement it because of how much it speeds up the process of training, since we only have to worry about the final layer. An added benefit of using this approach is that it avoids overfitting to our data, which would have posed a significant challenge had we trained our own CNN from scratch, since each class only has 150 images.

ResNet-18's original final layer performs Linear(512, 1000), which maps 512 feature values to 1,000 ImageNet classes. We replaced it with Linear(512, 50) to match our 50-breed output. This new layer is randomly initialised and is the only layer trained during our experiments.

The output of this layer is 50 raw scores (logits), one per breed. The loss function and prediction code later convert them to probabilities as needed.

Training and Results

The model tends to converge within 10-15 epochs, so we standardized it to run for 15 during training. After completion, the parameters of the epoch with the best results are saved and used for testing the model and generating graphs.

During training, two main metrics are used to evaluate each epoch: Top-1 score, which is the model's accuracy for correctly identifying the test image's breed as the most likely match, and the Top-3 score, which is the model's accuracy for having the breed within the top 3 matches.

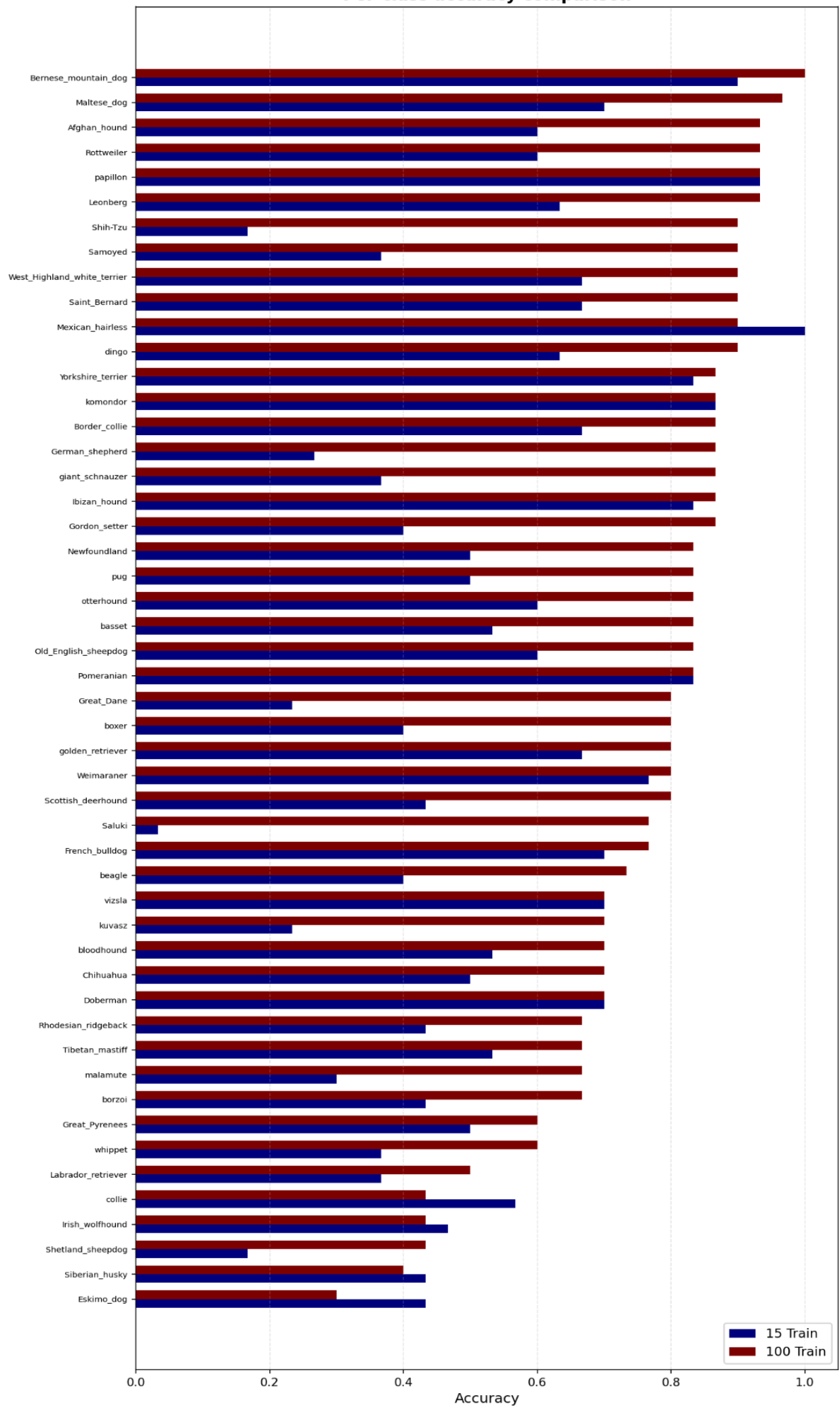
In the previous milestone, we used linear regression to classify 3 distinct dog breeds, which had an average accuracy of 0.578. After training, this model had a 0.22 overall increase, with an average top-1 accuracy of 0.80. The 3 chosen dog breeds for linear regression were: Italian greyhound, Mexican Hairless, and Pomeranian. For experimental purposes, the Italian greyhound was not included as one of the 50 selected dog breeds. To further test the new CNN model, we used 3 test images, one for each of the breeds used by the benchmarking model (including the greyhound, despite it not being a possible output). Since the breeds were originally picked for being very distinct, the model performed spectacularly for the two breeds that the model trained on, while for the third breed, the results were a lot more spread out. The exact results for this can be seen in the proof of accomplishment section.

Proof of accomplishment

Per-class accuracy comparison

The first plot shows the accuracy for each breed, with 2 graphs for each: The accuracy when using 15 images to train (135 for testing), and the accuracy when using 100 images to train (50 for testing). This graph helps us compare our results to the baseline results of the creators of the Stanford Dogs Dataset, since they have an identical graph for their 120 breeds. This demonstrates our progress from milestone 1, where we got similar results to now, where we outperform them for most of the breeds by a large margin.

Per-class accuracy comparison



Model Training Output

This screenshot showcases the output of the model during training, demonstrating it loading the dataset and updating the best results as well as a summary report for each epoch. This shows the progress done in our data and model.

```
=====
STEP 1 – Loading dataset
=====
Dataset loaded from: selected_breeds
Total images : 7500
Training      : 5625 (75%)
Test         : 1875 (25%)
Classes      : 50 breeds
Batch size   : 32

=====
STEP 2 – Building model
Using device: cuda
Model ready.
Total parameters      : 11,202,162
Trainable parameters : 25,650 (0.2% of total)
Output classes       : 50

=====
STEP 3 – Training for 15 epoch(s)
=====
Epoch 1/15 | Train loss: 2.2489 Train acc: 47.82% | Test loss: 1.2090 Top-1: 70.35% Top-3: 90.40%
            New best Top-1 accuracy – checkpoint saved
Epoch 2/15 | Train loss: 1.0952 Train acc: 72.27% | Test loss: 0.9102 Top-1: 73.76% Top-3: 92.27%
            New best Top-1 accuracy – checkpoint saved
Epoch 3/15 | Train loss: 0.8505 Train acc: 76.50% | Test loss: 0.7657 Top-1: 77.07% Top-3: 93.23%
            New best Top-1 accuracy – checkpoint saved
Epoch 4/15 | Train loss: 0.7411 Train acc: 79.54% | Test loss: 0.7140 Top-1: 78.61% Top-3: 93.07%
            New best Top-1 accuracy – checkpoint saved
Epoch 5/15 | Train loss: 0.6743 Train acc: 80.55% | Test loss: 0.6968 Top-1: 79.09% Top-3: 93.07%
            New best Top-1 accuracy – checkpoint saved
Epoch 6/15 | Train loss: 0.5958 Train acc: 83.47% | Test loss: 0.6682 Top-1: 78.88% Top-3: 93.71%
Epoch 7/15 | Train loss: 0.5858 Train acc: 83.89% | Test loss: 0.6443 Top-1: 80.05% Top-3: 93.97%
            New best Top-1 accuracy – checkpoint saved
Epoch 8/15 | Train loss: 0.5734 Train acc: 84.37% | Test loss: 0.6728 Top-1: 78.99% Top-3: 93.81%
Epoch 9/15 | Train loss: 0.5610 Train acc: 84.14% | Test loss: 0.6526 Top-1: 79.47% Top-3: 93.87%
Epoch 10/15 | Train loss: 0.5676 Train acc: 84.23% | Test loss: 0.6504 Top-1: 79.41% Top-3: 93.97%
Epoch 11/15 | Train loss: 0.5452 Train acc: 85.12% | Test loss: 0.6461 Top-1: 80.00% Top-3: 94.13%
Epoch 12/15 | Train loss: 0.5563 Train acc: 84.48% | Test loss: 0.6547 Top-1: 79.52% Top-3: 94.13%
Epoch 13/15 | Train loss: 0.5597 Train acc: 84.60% | Test loss: 0.6519 Top-1: 79.84% Top-3: 94.24%
Epoch 14/15 | Train loss: 0.5547 Train acc: 84.91% | Test loss: 0.6337 Top-1: 80.37% Top-3: 94.19%
            New best Top-1 accuracy – checkpoint saved
Epoch 15/15 | Train loss: 0.5453 Train acc: 84.98% | Test loss: 0.6417 Top-1: 80.05% Top-3: 93.97%
```

Benchmark Breeds test results

This screenshot shows the results of the test discussed at the end of the previous section, which saw us test the model on images of the breeds used for the benchmarking model in milestone 1. We also added an extra breed to further test the model (German Shepherd). From top to bottom: GS.jpg is the result for an image of a german shepherd, the I_G.jpg is the result for the Italian greyhound, the M.jpg is the result for the Mexican Hairless, and the P.jpg is the result for the Pomeranian. This screenshot demonstrates our progress towards our final product: A model that outputs the top 3 most likely breeds for a provided image.

```
Imagen: images/GS.jpg
1. German_shepherd      83.12%
2. Leonberg             8.47%
3. Afghan_hound         2.45%

Imagen: images/I_G.jpg
1. whippet              46.76%
2. Ibizan_hound         41.87%
3. Saluki                2.61%

Imagen: images/M.jpg
1. Mexican_hairless     91.00%
2. Great_Dane           3.20%
3. whippet              3.10%

Imagen: images/P.jpg
1. Pomeranian           99.00%
2. papillon             0.51%
3. Chihuahua            0.26%
```

Challenges

- Training the model was incredibly slow on CPU, taking over 30 minutes to complete. This was sped up considerably by switching to Cuda (around 10 minutes to complete), but was still a hassle to work around.
- Before the image names were cleaned to remove the Stanford labels that preceded the actual breed, the obtained top-1 and top-3 accuracies were hovering around 1-3%. This was caused by the pytorch library treating the number as part of the class, which created many arbitrary classes and muddled results.
- We originally wanted to have a fully functional interface which allows users to upload an image of their own, and then run the model on that image, but this is taking more time than expected, and is not yet ready to showcase for this report.

Project Changes

- In the original proposal, we planned on training our own CNN from scratch. After instructor feedback and deliberation between our group, we instead opted to use transfer learning on a pre-trained model. This significantly cut down the scope of the project, and made the amount of time we would have had to spend on this project much more reasonable for the timeframe and schedules we are working with.
- In Milestone 1, we used an 80/20 training-testing split, but after experimentation, we reverted back to the original value in our project proposal of 75/25. This change was done because the results turned out better with the original split for this model.

Appendix A: Chosen Dog Breeds

There were 2 main metrics used to narrow down the dog breeds from 120 to 50: How distinct the breed is and its popularity.

The full list Chosen dog breeds:

Afghan hound, Basset Hound, Beagle, Bernese Mountain Dog, Bloodhound, Border Collie, Borzoi, Boxer, Chihuahua, Collie, Dingo, Doberman, Eskimo dog, French Bulldog, German Shepherd, Giant schnauzer, Golden Retriever, Gordon setter, Great Dane, Great Pyrenees, Ibizan hound, Irish wolfhound, Komondor, Kuvasz, Labrador Retriever, Leonberg, Malamute, Maltese, Mexican hairless, Newfoundland, Old English sheepdog, Otterhound, Papillon, Pomeranian, Pug, Rhodesian Ridgeback, Rottweiler, Saint Bernard, Saluki, Samoyed, Scottish deerhound, Shetland Sheepdog, Shih-Tzu, Siberian Husky, Tibetan mastiff, Vizsla, Weimaraner, West Highland White Terrier, Whippet, Yorkshire Terrier

Appendix B: Image Transform Pipelines

Two separate pipelines are used, one for the training data and one for the test data.

The training pipeline does the following steps in order:

Resize(224, 224)	This scales every image to exactly 224×224 pixels. This is done because ResNet-18 requires this fixed input size because its architecture ends with a fixed-size fully-connected layer.
RandomHorizontalFlip	mirrors the image left-to-right with a 50% probability. A dog facing left and a dog facing right are the same breed, so this step is done to teach the model that horizontal orientation is not a discriminating feature.
RandomRotation(15)	Randomly rotates the image by a uniformly sampled angle between -15° and $+15^\circ$. Dogs in photos are rarely perfectly upright; this makes the model more robust to slight tilts in the input image.
ToTensor	PIL Image $\rightarrow$ torch.FloatTensor of shape [3, 224, 224]. Pixel values are rescaled from the integer range [0, 255] to floats in [0.0, 1.0].
Normalize(mean, std)	Shifts and scales each colour channel using ImageNet statistics (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]). This is required because ResNet-18's pretrained weights were optimised for this exact pixel distribution. Skipping normalisation would cause the frozen feature layers to produce meaningless activations.

The test data has no need for the rotation and flip functions, so it only calls Resize, ToTensor and Normalize, all using the same parameters as described above. This is done to ensure that evaluation results are not influenced by the random transformations, which allows us to compare them directly across different iterations.